

Recipe Search Fields

All fields are derived from the recipe JSON file. There are **5 searchable fields**.

1. recipe body

Where: The full recipe content in the JSON file — all steps with their "provider", "name", and "input" fields, preserving step order and nesting.

Example: This is the cleaned recipe body by our team.

```
Connectors: salesforce, workato_db_table, slack_bot
Steps:
- trigger: salesforce/updated_record
  fields: object, since
- action: workato_db_table/upsert_record
  fields: table_id, record_id
- action: slack_bot/post_message
  fields: channel, message
```

Used for: full-text search and keyword search. This is the primary search surface.

2. provider — Integration/app names

Where: The "provider" field on each step in the recipe JSON (present at every level of nesting). We collect all unique values across the recipe.

Examples: salesforce, workato_db_table, slack_bot, google_sheets, jira

Used for: matching recipes by app name, with tolerance for typos. Highest priority signal when searching by integration.

3. actions — Specific operations performed

Where: The combination of "provider" and "name" fields on each step in the recipe JSON (e.g. salesforce/search_records).

Examples: salesforce/search_records, workato_db_table/upsert_record, slack_bot/post_message

Used for: matching recipes by what operation is performed, with tolerance for typos. More specific than provider.

4. input_fields — Step configuration keys

Where: The input parameter names configured on each action step in the recipe JSON.

Examples: `table_id`, `record_id`, `continuation_token`, `object_type`

Used for: matching recipes by how steps are configured, with tolerance for typos.

5. `datapill_fields` — Step output schema field names

Where: The output field names each step exposes in the recipe JSON — the data that subsequent steps can consume.

Examples: `record`, `records`, `continuation_token`, `package_version_name`

Used for: matching recipes by the data they produce, with tolerance for typos. Lower priority than other fields since output schemas tend to be more generic.

6. embedding vector — Semantic representation of the recipe body

Where: Generated from the recipe body (item 1) by passing it through an embedding model.

Used for: dense (semantic) search — matching by meaning rather than exact keywords. For example, a query like "sync customer data" can match recipes that use Salesforce and a database without sharing any words.

Field Hierarchy Summary

Field	Source in Recipe	Search Role	Specificity
<code>recipe body</code>	Full recipe content from JSON	Full-text search, keyword	Broadest
<code>embedding vector</code>	Vector encoding of the recipe body	Dense (semantic) search	Broadest
<code>provider</code>	" <code>provider</code> " field on each step (all unique values)	Fuzzy match	App-level
<code>actions</code>	" <code>provider</code> " + " <code>name</code> " fields on each step	Fuzzy match	Operation-level
<code>input_fields</code>	Input parameter names configured on each step	Fuzzy match	Config-level
<code>datapill_fields</code>	Output field names each step exposes	Fuzzy match	Output-level